SQL JOIN and GROUP BY Lab - Student Exercises

Lab Introduction

This lab is designed to help you practice SQL JOINs and GROUP BY clauses through hands-on exercises. You will work with a company database containing information about departments, employees, customers, orders, products, and order details.

Database Schema Overview

The database contains the following tables:

departments

- dept_id (Primary Key)

- dept_name

- location

```sql
INSERT INTO departments VALUES
(1,'HR','Hyderabad'),(2,'IT','Bangalore'),(3,'Finance','Mumbai'),
(4,'Sales','Delhi'),(5,'Marketing','Chennai'),
(6,'Support','Pune'),(7,'R&D','Bangalore'),
(8,'Admin','Hyderabad'),(9,'Operations','Mumbai'),
(10,'Security','Delhi'),(11,'QA','Pune'),
(12,'Design','Chennai'),(13,'Legal','Mumbai'),
(14,'Training','Hyderabad'),(15,'Logistics','Delhi'),
(16,'Procurement','Chennai'),(17,'Analytics','Bangalore'),
(18,'Cloud','Hyderabad'),(19,'AI','Bangalore'),
(20,'ML','Hyderabad'),(21,'Infra','Pune'),
(22,'Compliance','Delhi'),(23,'Audit','Mumbai'),
(24,'Content','Chennai'),(25,'Media','Delhi'),
(26,'Research','Bangalore'),(27,'Innovation','Hyderabad'),
(28,'Strategy','Mumbai'),(29,'CSR','Delhi'),
(30,'Quality','Pune');
```

employees

- employee_id (Primary Key)
- first_name
- last_name
- email
- hire_date
- salary
- dept_id (Foreign Key → departments)
- manager_id (Foreign Key → employees)

```sql
INSERT INTO employees VALUES
(1,'Amit','Sharma','amit@c.com','2019-01-10',95000,2,NULL),
(2,'Ravi','Kumar','ravi@c.com','2020-02-15',72000,2,1),
(3,'Neha','Singh','neha@c.com','2021-03-12',68000,1,1),
(4,'Pooja','Mehta','pooja@c.com','2018-04-18',98000,3,NULL),
(5,'Rahul','Verma','rahul@c.com','2019-05-20',76000,4,4),
(6,'Anita','Rao','anita@c.com','2023-06-01',56000,1,3),
(7,'Suresh','Patel','suresh@c.com','2021-07-11',93000,5,4),
(8,'Kiran','Das','kiran@c.com','2017-08-21',105000,7,NULL),
(9,'Manoj','Iyer','manoj@c.com','2022-09-10',61000,6,8),
(10,'Divya','Nair','divya@c.com','2023-10-05',59000,6,8),
(11,'Arjun','Malik','arjun@c.com','2021-11-09',69000,4,5),
(12,'Sneha','Joshi','sneha@c.com','2018-12-01',83000,3,4),
(13,'Vikas','Gupta','vikas@c.com','2020-01-19',94000,5,7),
(14,'Ritu','Kapoor','ritu@c.com','2022-02-14',78000,2,1),
(15,'Nikhil','Bansal','nikhil@c.com','2024-01-05',52000,1,3),
(16,'Sameer','Ali','sameer@c.com','2020-03-11',90000,8,1),
(17,'Priya','Sen','priya@c.com','2021-04-12',88000,9,NULL),
(18,'Mohit','Jain','mohit@c.com','2022-05-14',67000,9,17),
(19,'Kavya','Reddy','kavya@c.com','2020-06-16',92000,10,NULL),
```

customers

- customer_id (Primary Key)

- customer_name

- email

- city

- country

```sql
INSERT INTO customers VALUES
(1,'Alice','a@c.com','Delhi','India'),
(2,'Bob','b@c.com','Mumbai','India'),
(3,'Charlie','c@c.com','London','UK'),
(4,'David','d@c.com','Berlin','Germany'),
(5,'Eva','e@c.com','Paris','France'),
(6,'Frank','f@c.com','Rome','Italy'),
(7,'Grace','g@c.com','Madrid','Spain'),
(8,'Henry','h@c.com','Delhi','India'),
(9,'Irene','i@c.com','NY','USA'),
(10,'Jack','j@c.com','LA','USA'),
(11,'Kevin','k@c.com','Toronto','Canada'),
(12,'Lily','l@c.com','Tokyo','Japan'),
(13,'Mona','m@c.com','Sydney','Australia'),
(14,'Nina','n@c.com','Dubai','UAE'),
(15,'Oscar','o@c.com','Cape Town','South Africa'),
(16,'Paul','p@c.com','Delhi','India'),
(17,'Quinn','q@c.com','Mumbai','India'),
(18,'Rose','r@c.com','Pune','India'),
(19,'Steve','s@c.com','Chennai','India'),
(20,'Tina','t@c.com','Bangalore','India'),
(21,'Uma','u@c.com','Hyderabad','India'),
```

orders

- order_id (Primary Key)

- customer_id (Foreign Key → customers)

- order_date

- total_amount

- status

```sql
INSERT INTO orders VALUES
(1,1,'2024-01-10',1200,'Completed'),
(2,1,'2024-02-15',800,'Completed'),
(3,2,'2024-03-20',1500,'Completed'),
(4,3,'2024-04-25',2000,'Pending'),
(5,4,'2024-05-30',900,'Completed'),
(6,5,'2024-06-10',400,'Completed'),
(7,6,'2024-07-05',1100,'Completed'),
(8,7,'2024-08-18',600,'Cancelled'),
(9,8,'2024-09-12',1300,'Completed'),
(10,9,'2024-10-22',1700,'Completed'),
(11,10,'2024-11-01',950,'Completed'),
(12,11,'2024-11-15',500,'Completed'),
(13,12,'2024-12-05',2100,'Completed'),
(14,13,'2024-12-10',700,'Pending'),
(15,1,'2024-12-20',1600,'Completed'),
(16,16,'2024-03-11',900,'Completed'),
(17,17,'2024-04-12',1300,'Completed'),
(18,18,'2024-05-13',750,'Completed'),
(19,19,'2024-06-14',1800,'Completed'),
(20,20,'2024-07-15',600,'Completed'),
(21,21,'2024-08-16',1400,'Completed'),
(22,22,'2024-09-17',2200,'Completed'),
```

products

- product_id (Primary Key)
- product_name
- category
- unit_price

```sql
INSERT INTO products VALUES
(1,'Laptop','Electronics',600),
(2,'Phone','Electronics',400),
(3,'Tablet','Electronics',300),
(4,'Chair','Furniture',150),
(5,'Desk','Furniture',250),
(6,'Pen','Stationery',10),
(7,'Notebook','Stationery',20),
(8,'Printer','Electronics',200),
(9,'Scanner','Electronics',180),
(10,'Monitor','Electronics',220),
(11,'Mouse','Electronics',25),
(12,'Keyboard','Electronics',30),
(13,'Lamp','Furniture',90),
(14,'Bag','Accessories',70),
(15,'Headphones','Accessories',120),
(16,'Webcam','Electronics',80),
(17,'Router','Electronics',150),
(18,'Speaker','Electronics',200),
(19,'Microphone','Electronics',140),
(20,'SSD','Electronics',160),
(21,'Hard Disk','Electronics',110),
```

****order_details****

- order_detail_id (Primary Key)

- order_id (Foreign Key → orders)

- product_id (Foreign Key → products)

- quantity

- unit_price

```sql
INSERT INTO order_details VALUES
(1,1,1,1,600),(2,1,2,1,400),
(3,2,3,2,300),(4,2,4,1,150),
(5,3,5,2,250),(6,4,6,10,10),
(7,5,7,5,20),(8,6,8,2,200),
(9,7,9,1,180),(10,8,10,3,220),
(11,9,11,4,25),(12,10,12,2,30),
(13,11,13,1,90),(14,12,14,5,70),
(15,13,15,2,120),(16,14,16,3,80),
(17,15,17,1,150),(18,1,3,1,300),
(19,1,4,1,150),(20,18,20,2,160),
(21,19,21,3,110),(22,20,22,4,90),
(23,21,1,2,600),(24,22,2,1,400),
(25,23,3,3,300),(26,24,4,2,150),
(27,25,27,1,500),(28,26,28,5,20),
(29,27,29,3,35),(30,28,30,1,130);
```

Setup Instructions

Before starting the exercises, run the setup script provided by your trainer to create the database tables and insert sample data.

Exercises

Exercise 1: Basic INNER JOIN

List all employees with their department names. Display employee ID, first name, last name, and department name. Order the results by employee ID.

Hint: You need to join the employees and departments tables.

-- Write your query here

Expected columns: employee_id, first_name, last_name, dept_name

```
-- Shaik Shanaaz
-- Exercise 1: Basic INNER JOIN
-- selecting employee details
SELECT employee_id, first_name, last_name, dept_name
-- joining two tables by using inner join
FROM employees e JOIN departments d
 -- joining based on matching dept_id in both tables
ON e.dept_id = d.dept_id
-- arranging result in order of employee_id
ORDER BY employee_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	first_name	last_name	dept_name
1	Amit	Sharma	IT
2	Ravi	Kumar	IT
3	Neha	Singh	HR
4	Pooja	Mehta	Finance
5	Rahul	Verma	Sales
6	Anita	Rao	HR
7	Suresh	Patel	Marketing
8	Kiran	Das	R&D
9	Manoj	Iyer	Support
10	Divya	Nair	Support
11	Arjun	Malik	Sales
12	Sneha	Joshi	Finance
13	Vikas	Gupta	Marketing
14	Ritu	Kapoor	IT
15	Nikhil	Bansal	HR
16	Sameer	Ali	Admin
17	Priya	Sen	Operations
18	Mohit	Jain	Operations

Exercise 2: LEFT JOIN to Find Missing Data

Find all customers and their order count, including customers who haven't placed any orders. Display customer ID, customer name, and the count of orders. Order by order count in descending order.

Hint: Use LEFT JOIN to include customers with no orders, and COUNT with GROUP BY.

-- Write your query here

Expected columns: customer_id, customer_name, order_count

```
-- Shaik Shanaaz
-- Exercise 2: LEFT JOIN to Find Missing Data

-- selects customer details and count of their orders
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS order_count
-- linking two tables by using left join
FROM customers AS c LEFT JOIN orders AS o
-- join by taking all customer id's in left table and match customer id's in right
ON c.customer_id = o.customer_id
-- group by customers with their no.of orders
GROUP BY customer_id, customer_name
-- arrange result with highest order count
ORDER BY order_count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	order_count
1	Alice	3
2	Bob	1
3	Charlie	1
4	David	1
5	Eva	1
6	Frank	1
7	Grace	1
8	Henry	1
9	Irene	1
10	Jack	1
11	Kevin	1
12	Lily	1
13	Mona	1
16	Paul	1
17	Quinn	1
18	Rose	1

Exercise 3: Multiple JOINs

List all orders with customer names and the total number of items in each order. Display order ID, customer name, order date, total amount, count of different items, and sum of quantities. Order by order ID.

Hint: You need to join three tables: orders, customers, and order_details.

-- Write your query here

Expected columns: order_id, customer_name, order_date, total_amount, item_count, total_quantity

```sql
-- NAME: Shaik Shanaaz
-- Exercise 3: Multiple JOINs

-- selects order details
SELECT o.order_id, c.customer_name, o.order_date, o.total_amount,
-- uses aggregate functions to get count and sum
COUNT(od.product_id) AS item_count, SUM(od.quantity) AS total_quanity
-- join order and customer table using inner join based on customer_id
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
-- join order and order_details table based on order_id
JOIN order_details od ON o.order_id = od.order_id
-- group by no.of products and sum of quantity
GROUP BY o.order_id, c.customer_name, o.order_date, o.total_amount
ORDER BY o.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_name	order_date	total_amount	item_count	total_quanity
1	Alice	2024-01-10	1200.00	2	2
2	Alice	2024-02-15	800.00	2	3
3	Bob	2024-03-20	1500.00	1	2
4	Charlie	2024-04-25	2000.00	1	10
5	David	2024-05-30	900.00	1	5
6	Eva	2024-06-10	400.00	1	2
7	Frank	2024-07-05	1100.00	1	1
8	Grace	2024-08-18	600.00	1	3
9	Henry	2024-09-12	1300.00	1	4
10	Irene	2024-10-22	1700.00	1	2
11	Jack	2024-11-01	950.00	1	1
12	Kevin	2024-11-15	500.00	1	5
13	Lily	2024-12-05	2100.00	1	2
14	Mona	2024-12-10	700.00	1	3

Exercise 4: GROUP BY with Aggregates

Calculate the average salary by department. Display department name, count of employees, average salary, minimum salary, and maximum salary. Order by average salary in descending order.

Hint: Join departments and employees, then use GROUP BY with aggregate functions.

-- Write your query here

Expected columns: dept_name, employee_count, avg_salary, min_salary, max_salary

```sql
-- ### Exercise 4: GROUP BY with Aggregates.
-- select department name
SELECT d.dept_name,
    -- no.of employees
    COUNT(e.employee_id) AS employee_count,
    -- average salary
    AVG(e.salary) AS avg_salary,
    -- minimum salary
    MIN(e.salary) AS min_salary,
    -- maximum salary
    MAX(e.salary) AS max_salary
-- join department and employees table
FROM departments d JOIN employees e
ON d.dept_id = e.dept_id
-- group all data by dept_name
GROUP BY d.dept_name
-- arrange result by highest average salary
ORDER BY avg_salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	employee_count	avg_salary	min_salary	max_salary
Legal	1	74000.0000	74000	74000
Sales	2	72500.0000	69000	76000
Logistics	2	70500.0000	69000	72000
Admin	1	70000.0000	70000	70000
Marketing	2	68500.0000	64000	73000
Design	1	66000.0000	66000	66000
Analytics	1	62000.0000	62000	62000
Support	2	60000.0000	59000	61000
HR	3	58666.6667	52000	68000

Exercise 5: HAVING Clause

Find departments with an average salary greater than 65000 and more than 2 employees. Display department name, employee count, and average salary. Order by average salary in descending order.

Hint: Use HAVING clause to filter groups after aggregation.

Expected columns: dept_name, employee_count, avg_salary

-- Write your query here

```sql
# NAME: Shaik Shanaaz
## Exercise 5: HAVING Clause

-- select department name
SELECT d.dept_name,
    -- no.of employees
    COUNT(e.employee_id) AS employee_count,
    -- average salary of employees
    AVG(e.salary) AS avg_salary
-- join departments and employee table
FROM departments d JOIN employees e
ON d.dept_id=e.dept_id
-- group data by departemt
GROUP BY d.dept_name
-- having salary >65000 and no.of employees more than 2
HAVING avg_salary > 65000 AND COUNT(employee_id) > 2
-- arrange result by the highest average salary
ORDER BY avg_salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	employee_count	avg_salary
IT	3	81666.6667

Exercise 6: SELF JOIN

List all employees with their manager's name. Display employee ID, employee full name, employee salary, manager full name, and manager salary. Order by employee ID.

Hint: Join the employees table to itself. Use LEFT JOIN to include employees without managers.

-- Write your query here

Expected columns: employee_id, employee_name, employee_salary, manager_name, manager_salary

```sql
-- NAME: Shaik Shanaaz
-- Exercise 6: SELF JOIN

SELECT e.employee_id,
    -- using CONCAT function to get employee fullname
    CONCAT(e.first_name,' ',e.last_name) AS employee_name,
    -- get employee salary
    e.salary AS employee_salary,
    -- using CONCAT function to get manager fullname
    CONCAT(m.first_name, ' ', m.last_name) AS manager_name,
    -- get manager salary
    m.salary AS manager_salary
-- self join employees table to get manager details
FROM employees e LEFT JOIN employees m
ON e.employee_id = m.employee_id
-- arrange result order to employee_id
ORDER BY e.employee id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	employee_salary	manager_name	manager_salary
1	Amit Sharma	95000	Amit Sharma	95000
2	Ravi Kumar	72000	Ravi Kumar	72000
3	Neha Singh	68000	Neha Singh	68000
4	Pooja Mehta	98000	Pooja Mehta	98000
5	Rahul Verma	76000	Rahul Verma	76000
6	Anita Rao	56000	Anita Rao	56000
7	Suresh Patel	93000	Suresh Patel	93000
8	Kiran Das	105000	Kiran Das	105000
9	Manoj Iyer	61000	Manoj Iyer	61000
10	Divya Nair	59000	Divya Nair	59000
11	Arjun Malik	69000	Arjun Malik	69000

Exercise 7: Complex Aggregation

Find the total revenue by product category. Display category, count of distinct orders, total units sold, and total revenue. Order by total revenue in descending order.

Hint: Join products and order_details, calculate revenue as quantity × unit_price.

-- Write your query here

Expected columns: category, order_count, total_units_sold, total_revenue

```sql
-- NAME: Shaik Shanaaz
--  Exercise 7: Complex Aggregation

-- select category from product table
SELECT p.category,
    -- no.of distinct orders
    COUNT(DISTINCT od.order_id) AS order_count,
    -- total unit sold
    SUM(od.unit_price) AS total_units_sold,
    -- calculate total revenue
    COUNT(od.quantity * od.unit_price) AS total_revenue
-- join order_details and products table
FROM order_details od JOIN products p
-- group data by category
GROUP BY p.category
-- arrange result by highest total revenue
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	order_count	total_units_sold	total_revenue
Electronics	28	84690.00	540
Accessories	28	23525.00	150
Furniture	28	18820.00	120
Stationery	28	14115.00	90

Exercise 8: JOINs with WHERE and HAVING

Find customers who have placed more than 1 completed order and their total spending. Display customer ID, customer name, count of completed orders, and total spent. Order by total spent in descending order.

Hint: Filter for completed orders using WHERE, then use HAVING to filter groups with more than 1 order.

-- Write your query here

Expected columns: customer_id, customer_name, completed_orders, total_spent

```sql
-- NAME : Shaik Shanaaz
-- Exercise 8: JOINs with WHERE and HAVING

-- select customer details
SELECT c.customer_id, c.customer_name,
    -- no.of complete orders
    COUNT(o.order_id) AS completed_orders,
    -- total amount spent
    COUNT(o.total_amount) AS total_spent
-- join customer and orders table
FROM customers c JOIN orders o
ON c.customer_id = o.customer_id
-- taking only those data where the order status is "completed"
WHERE o.status = "completed"
-- group data by complete orders and total spents
GROUP BY c.customer_id, c.customer_name
-- and having no.of orders more than 1
HAVING COUNT(o.order_id) > 1
-- arrange result by highest total spent
ORDER BY total_spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	completed_orders	total_spent
1	Alice	3	3

Exercise 9: Finding Unmatched Records

Find all products that have never **been** ordered. Display product ID, product name, category, and unit price. Order by product ID.

Hint: Use LEFT JOIN and check for NULL values in the joined table.

-- Write your query here

Expected columns: product_id, product_name, category, unit_price

```
-- NAME : Shaik Shanaaz
-- Exercise 9: Finding Unmatched Records

-- select product that have never use
-- selects product details
SELECT p.product_id, p.product_name, p.category, p.unit_price
-- join products and order_details table
FROM products p LEFT JOIN order_details od
ON p.product_id = od.product_id
-- where product is neve ordered
WHERE od.product_id  IS NULL
-- arrange result by product id
ORDER BY p.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	category	unit_price
23	Stylus	Accessories	60.00
24	Whiteboard	Furniture	300.00
25	File Folder	Stationery	15.00
26	Calculator	Electronics	50.00

Exercise 10: Advanced Multi-Table Analysis

For each country, show the total number of orders, total revenue, and average order value. Only include countries that have orders. Display country, total orders, customer count, total revenue, and average order value. Order by total revenue in descending order.

Hint: Join customers and orders, group by country, and use HAVING to exclude countries with no orders.

-- Write your query here

Expected columns: country, total_orders, customer_count, total_revenue, avg_order_value

```sql
-- NAME : Shaik Shanaaz
-- Exercise 10: Advanced Multi-Table Analysis

-- select country wise order and revenue
SELECT c.country,
    COUNT(o.order_id) AS total_order,
    COUNT(DISTINCT c.customer_id) as customer_count,
    SUM(o.total_amount) AS total_revenue,
    AVG(o.total_amount) AS avg_order_value
-- join customers and orders table
FROM customers c JOIN orders o
ON c.customer_id = o.customer_id
-- group data by country
GROUP BY c.country
-- countries which having orders
HAVING COUNT(o.order_id)>0
-- arrange by highest total revenue
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	total_order	customer_count	total_revenue	avg_order_value
India	13	11	15650.00	1203.846154
USA	4	4	4350.00	1087.500000
UK	2	2	4200.00	2100.000000
Japan	2	2	4100.00	2050.000000
Germany	2	2	1850.00	925.000000
France	2	2	1650.00	825.000000
Canada	2	2	1400.00	700.000000
Italy	1	1	1100.00	1100.000000
Australia	1	1	700.00	700.000000

Exercise 11: Subquery with GROUP BY

Find departments where the average salary is above the company average. Display department name, department average salary, and company average salary. Order by department average salary in descending order.

Hint: Use a subquery in the HAVING clause to compare against the overall average. --

Write your query here

Expected columns: dept_name, dept_avg_salary, company_avg_salary

```sql
-- NAME : Shaik Shanaaz
-- Exercise 11: Subquery with GROUP BY
-- subquery won't support GROUP BY clause

SELECT d.dept_name,
    -- department average salary
    AVG(e.salary) AS dept_avg_salary,
    -- using sub query to get company average salary
    (SELECT AVG(salary) FROM employees) as company_avg_salary
-- join departments and employees table as subquery don't support GROUP BY clause
FROM departments d JOIN employees e
ON d.dept_id = e.dept_id
-- Group data by department
GROUP BY d.dept_name
-- departments having average salary > company's average salary
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees)
-- arrange result by highest department average salary
ORDER BY dept_avg_salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	dept_avg_salary	company_avg_salary
R&D	105000.0000	74966.6667
Marketing	93500.0000	74966.6667
Finance	90500.0000	74966.6667
Admin	90000.0000	74966.6667
IT	81666.6667	74966.6667
Training	81000.0000	74966.6667
QA	78500.0000	74966.6667
Operations	77500.0000	74966.6667
Security	76000.0000	74966.6667

Exercise 12: Date-Based Grouping

Show monthly order statistics for 2024. Display month (YYYY-MM format), order count, monthly revenue, and average order value. Order by month.

Hint: Use date functions to extract year-month. Functions vary by database:

- SQLite: `strftime('%Y-%m', order_date)`

- MySQL: `DATE_FORMAT(order_date, '%Y-%m')`

- PostgreSQL: `TO_CHAR(order_date, 'YYYY-MM')`

-- Write your query here

Expected columns: month, order_count, monthly_revenue, avg_order_value

```sql
-- NAME : Shaik Shanaaz
-- Exercise 12: Date-Based Grouping

-- selects monthly order statistics for 2024 using Date_Format
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
    -- no.of orders
    COUNT(order_id) AS order_count,
    -- monthly revenue
    SUM(total_amount) AS monthly_revenue,
    -- average of orders amounts
    AVG(total_amount) AS avg_order_value
FROM orders
-- group data by date_format
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
# Arrange result by month
ORDER BY month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	order_count	monthly_revenue	avg_order_value
2024-01	2	2000.00	1000.000000
2024-02	2	1900.00	950.000000
2024-03	3	3000.00	1000.000000
2024-04	3	5300.00	1766.666667
2024-05	3	2550.00	850.000000
2024-06	2	2200.00	1100.000000
2024-07	2	1700.00	850.000000
2024-08	2	2000.00	1000.000000
2024-09	2	3500.00	1750.000000
2024-10	2	2650.00	1325.000000
2024-11	3	2700.00	900.000000
2024-12	4	6100.00	1525.000000

Exercise 13: Complex Filtering

Find the top 3 products by revenue. Display product ID, product name, category, total quantity sold, total revenue, and times ordered. Order by total revenue in descending order and limit to 3 results.

Hint: Calculate revenue as quantity × unit_price, use ORDER BY and LIMIT.

-- Write your query here

Expected columns: product_id, product_name, category, total_quantity, total_revenue, times_ordered

```sql
-- NAME : Shaik Shanaaz
-- Exercise 13: Complex Filtering

-- Select top 3 products based on total revenue
-- select product details
SELECT p.product_id, p.product_name, p.category,
    -- total quantity sold
    SUM(od.quantity) AS total_quantity,
    -- calculate revenue as quntity x unit_price
    COUNT(od.quantity * od.unit_price) AS  total_revenue,
    -- no.of times the order made
    COUNT(DISTINCT od.order_id) AS times_ordered
-- join products and order_details table
FROM products p JOIN order_details od
ON p.product_id = od.product_id
-- group data by total_quantity, total_revenue and times order
GROUP BY p.product_id, p.product_name, p.category
-- arrange result by highest total revenue and limit to 3 products only
ORDER BY total_revenue DESC LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_id	product_name	category	total_quantity	total_revenue	times_ordered
1	Laptop	Electronics	3	2	2
2	Phone	Electronics	2	2	2
3	Tablet	Electronics	5	2	2

Exercise 14: CASE with GROUP BY

Categorize employees by salary range and count employees in each range by department. Salary ranges:

- Entry Level: Less than 60000

- Mid Level: 60000 to 74999

- Senior Level: 75000 and above

Display department name, salary range, employee count, and average salary. Order by department name and then average salary.

Hint: Use CASE statement to create salary ranges, include it in both SELECT and GROUP BY.

Expected columns: dept_name, salary_range, employee_count, avg_salary

```sql
-- NAME : Shaik Shanaaz
--  Exercise 14: CASE with GROUP BY

SELECT d.dept_name,
    -- case is use to classify employee into salary range
    CASE
        WHEN e.salary < 60000 THEN "Entry Level" -- ssalary Less than 60000
        WHEN e.salary BETWEEN 60000 AND 74999 THEN "Mid Level" -- salary between 60000 to 74999
        ELSE "Senior Level" -- salary 75000 and above
    -- END closes the case  block and name the result
    END AS salary_range,
    -- performs aggregation functions
    COUNT(e.employee_id) AS employee_count,
    AVG(e.salary) AS avg_salary
-- join departments and employees table
FROM departments d JOIN employees e
ON d.dept_id= e.dept_id
```

```
-- group data by deptment name and salary range
GROUP BY d.dept_name,
        -- CASE statements
        CASE
            WHEN e.salary < 60000 THEN "Entry Level"
            WHEN e.salary BETWEEN 60000 AND 74999 THEN "Mid Level"
            ELSE "Senior Level"
         END
-- arrange result by department name and average salary
ORDER BY d.dept_name, avg_salary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	salary_range	employee_count	avg_salary
Admin	Senior Level	1	90000.0000
Analytics	Mid Level	1	62000.0000
Cloud	Entry Level	1	54000.0000
Design	Mid Level	1	66000.0000
Finance	Senior Level	2	90500.0000
HR	Entry Level	2	54000.0000
HR	Mid Level	1	68000.0000
IT	Mid Level	1	72000.0000
IT	Senior Level	2	86500.0000
Legal	Mid Level	1	74000.0000
Logistics	Mid Level	2	70500.0000
Marketing	Senior Level	2	93500.0000
Operations	Mid Level	1	67000.0000
Operations	Senior Level	1	88000.0000
Procurement	Entry Level	1	58000.0000
QA	Mid Level	1	72000.0000

Exercise 15: Comprehensive Report

Create a comprehensive customer report for customers with completed orders. Display: customer ID, customer name, country, total orders, total spent, average order value, last order date, and first order date. Order by total spent in descending order.

Hint: Join customers and orders, filter for completed status, use multiple aggregate functions.

-- Write your query here

Expected columns: customer_id, customer_name, country, total_orders, total_spent, avg_order_value, last_order_date, first_order_date

```sql
-- NAME : Shaik Shanaaz
-- Exercise 15: Comprehensive Report
-- customers report with completed orders
SELECT c.customer_id, c.customer_name, c.country,
    -- perform multiple aggregate functions
    COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_spent,
    AVG(o.total_amount) AS avg_order_value,
    MAX(order_date) AS last_order_date,
    MIN(order_date) AS first_order_date
-- join customer and orders table
FROM customers c JOIN orders o
ON c.customer_id = o.customer_id
-- check for customers with complete orders
WHERE o.status = "completed"
-- GROUP data for customer
GROUP BY c.customer_id, c.customer_name, c.country
-- arrange result in highest total spent
ORDER BY total_spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	country	total_orders	total_spent	avg_order_value	last_order_date	first_order_date
1	Alice	India	3	3600.00	1200.000000	2024-12-20	2024-01-10
22	Victor	UK	1	2200.00	2200.000000	2024-09-17	2024-09-17
12	Lily	Japan	1	2100.00	2100.000000	2024-12-05	2024-12-05
29	Carl	Japan	1	2000.00	2000.000000	2024-04-24	2024-04-24
19	Steve	India	1	1800.00	1800.000000	2024-06-14	2024-06-14
9	Irene	USA	1	1700.00	1700.000000	2024-10-22	2024-10-22
25	Yash	India	1	1700.00	1700.000000	2024-12-20	2024-12-20
2	Bob	India	1	1500.00	1500.000000	2024-03-20	2024-03-20
21	Uma	India	1	1400.00	1400.000000	2024-08-16	2024-08-16
8	Henry	India	1	1300.00	1300.000000	2024-09-12	2024-09-12
17	Quinn	India	1	1300.00	1300.000000	2024-04-12	2024-04-12
24	Xena	France	1	1250.00	1250.000000	2024-11-19	2024-11-19
6 24	Frank	Italy	1	1100.00	1100.000000	2024-07-05	2024-07-05
27	Aaron	USA	1	1100.00	1100.000000	2024-02-22	2024-02-22
10	Jack	USA	1	950.00	950.000000	2024-11-01	2024-11-01
23	Will	Germany	1	950.00	950.000000	2024-10-18	2024-10-18
4	David	Germany	1	900.00	900.000000	2024-05-30	2024-05-30
16	Paul	India	1	900.00	900.000000	2024-03-11	2024-03-11
30	Daisy	Canada	1	900.00	900.000000	2024-05-25	2024-05-25
26	Zara	India	1	800.00	800.000000	2024-01-21	2024-01-21
18	Rose	India	1	750.00	750.000000	2024-05-13	2024-05-13
20	Tina	India	1	600.00	600.000000	2024-07-15	2024-07-15
28	Bella	USA	1	600.00	600.000000	2024-03-23	2024-03-23
11	Kevin	Canada	1	500.00	500.000000	2024-11-15	2024-11-15
5	Eva	France	1	400.00	400.000000	2024-06-10	2024-06-10

Challenge Exercises (Bonus)

These exercises are optional and more advanced. Attempt them if you finish the main exercises early.

Challenge 1: Employee Hierarchy Analysis

Show each employee's level in the organization hierarchy and their path from the top. This requires a recursive query.

****Hint:**** Use a recursive CTE (Common Table Expression) with WITH RECURSIVE.

-- Write your query here

****Expected columns:**** employee_id, employee_name, manager_id, level, path

```sql
-- NAME : Shaik Shanaaz
-- Challenge 1: Employee Hierarchy Analysis

-- Create a recursive query to show employee hierarchy
WITH RECURSIVE emp_hierarchy AS (
    -- select top-level employees who does not have manager
    SELECT employee_id,
           CONCAT(first_name, ' ', last_name) AS employee_name,
           manager_id,
           1 AS level,
           CAST(CONCAT(first_name, ' ', last_name) AS CHAR(200)) AS path
    FROM employees
    WHERE manager_id IS NULL
    -- joins results of two SELECT queries
    UNION ALL

   -- attach employees to their managers level by level
    SELECT e.employee_id,
           -- Full name of employee
           CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
           e.manager_id,
```

```sql
    -- attach employees to their managers level by level
     SELECT e.employee_id,
            -- Full name of employee
            CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
            e.manager_id,
            eh.level + 1 AS level, -- -- Hierarchy level (1 = top level)
            -- path showing starting point of  hierarchy
            CONCAT(eh.path, " ->", e.first_name, ' ', e.last_name) AS path
     FROM employees e JOIN emp_hierarchy eh
     -- Match employee manager_id with previous level employee_id
      ON e.manager_id = eh.employee_id
)
-- displays final hierarchy result
SELECT employee_id,employee_name, manager_id, level, path
       FROM emp_hierarchy
-- arrange result by employee id
ORDER BY employee_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	manager_id	level	path
6	Anita Rao	3	3	Amit Sharma ->Neha Singh ->Anita Rao
7	Suresh Patel	4	2	Pooja Mehta ->Suresh Patel
8	Kiran Das	NULL	1	Kiran Das
9	Manoj Iyer	8	2	Kiran Das ->Manoj Iyer
10	Divya Nair	8	2	Kiran Das ->Divya Kiran Das ->Divya Nair
11	Arjun Malik	5	3	Pooja Mehta ->Ranjit Verma ->Arjun M...
12	Sneha Joshi	4	2	Pooja Mehta ->Sneha Joshi
13	Vikas Gupta	7	3	Pooja Mehta ->Suresh Patel ->Vikas G...
14	Ritu Kapoor	1	2	Amit Sharma ->Ritu Kapoor
15	Nikhil Bansal	3	3	Amit Sharma ->Neha Singh ->Nikhil Ba...
16	Sameer Ali	1	2	Amit Sharma ->Sameer Ali
17	Priya Sen	NULL	1	Priya Sen
18	Mohit Jain	17	2	Priya Sen ->Mohit Jain
19	Kavya Reddy	NULL	1	Kavya Reddy
20	Sanjay Das	19	2	Kavya Reddy ->Sanjay Das
21	Asha Menon	NULL	1	Asha Menon
22	Deepak Roy	21	2	Asha Menon ->Deepak Roy
23	Ramesh Naik	21	2	Asha Menon ->Ramesh Naik
24	Latha Krishnan	NULL	1	Latha Krishnan
25	Imran Khan	NULL	1	Imran Khan
26	Pankaj Mishra	25	2	Imran Khan ->Pankaj Mishra
27	Sonal Agarwal	25	2	Imran Khan ->Sonal Agarwal
28	Naveen Chopra	NULL	1	Naveen Chopra
29	Harsha V	28	2	Naveen Chopra ->Harsha V
30	Meena Paul	NULL	1	Meena Paul

Challenge 2: Product Affinity Analysis

Find products that are frequently bought together (appear in the same order at least twice). Display the two product names and how many times they were bought together.

Hint: Self-join order_details on order_id, ensure product_id1 < product_id2 to avoid duplicates.

-- Write your query here

Expected columns: product_1, product_2, times_bought_together

```sql
-- Challenge 2: Product Affinity Analysis
-- Find products that are bought together in the same order
-- select name of the first and second product
SELECT p1.product_name AS product_1,
       p2.product_name AS product_2,
       -- no.of times both products appear in same order
       COUNT(*) AS times_bought_together
-- Self join order_details on same order
FROM order_details od1 JOIN order_details od2
     ON od1.order_id = od2.order_id
     -- to avoid duplicate and reverse product pairs
    AND od1.product_id < od2.product_id
# Join to get product names
JOIN products p1 ON od1.product_id = p1.product_id  -- name of first product
JOIN products p2 ON od2.product_id = p2.product_id  -- name of second product
-- Group data by product pairs
GROUP BY p1.product_name, p2.product_name
-- only keep the product pairs bought together at least twice
HAVING COUNT(*) >= 2
# Arrange result by higher frequency
ORDER BY times_bought_together DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_1	product_2	times_bought_together
Tablet	Chair	2

Tips for Success

1. **Read the requirements carefully** - Make sure you understand what data is requested
2. **Start with the FROM clause** - Identify which tables you need
3. **Add JOINs one at a time** - Build your query incrementally 4. **Test your query** - Run it to see if you get the expected columns
5. **Check your results** - Do the numbers make sense?
6. **Use table aliases** - Makes queries more readable (e.g., `FROM employees e`)
7. **Format your SQL** - Use proper indentation and line breaks

8. **Comment your code** - Add comments to explain complex logic

Common Mistakes to Avoid

- Forgetting to include columns in GROUP BY that appear in SELECT
- Using WHERE instead of HAVING for filtering aggregated results
- Incorrect JOIN conditions leading to Cartesian products
- Not using LEFT JOIN when you need to include unmatched records - Forgetting to order results when specified

Submission Guidelines

1. Save your queries in a file named `firstname_lastname_sql_lab.sql`
2. Include comments with your name and the exercise number
3. Format your SQL code properly with indentation
4. Test all queries before submission
5. Submit your file to your trainer

Evaluation Criteria

Your work will be evaluated on:

- **Correctness (60%)** - Does the query return the expected results?
- **Query Structure (20%)** - Proper use of JOINs, GROUP BY, and HAVING?
- **Efficiency (10%)** - Is the query reasonably optimized?
- **Code Quality (10%)** - Is the SQL clean, readable, and well-formatted?

Good luck with your exercises!